

ApproxDPPoint::beta

Aishwarya Ramasethu (@aramasethu), Yu-Ju Ku (@yuju1998), Jordan Awan, Michael Shoemate (@Shoeboxam)

August 4, 2026

This proof resides in “contrib” because it has not completed the vetting process.

1 Hoare Triple

Precondition

Compiler-verified

- Argument `epsilon` of type `f64`
- Argument `delta` of type `f64`

Caller-verified

`epsilon` is finite and nonnegative, and `delta` is in $[0, 1]$.

Pseudocode

```
1 def approxdp_point_beta(  
2     epsilon: f64,  
3     delta: f64,  
4 ) -> Callable[[RBig], RBig]:  
5     epsilon = DInterval.point(epsilon)  
6     delta = RBig.try_from(delta)  
7  
8     exp_eps = epsilon.exp().upper.to_rbig() #  
9     exp_neg_eps = (-epsilon).exp().lower.to_rbig() #  
10  
11     def tradeoff(alpha: RBig) -> RBig: #  
12         t1 = RBig(1) - delta - exp_eps * alpha  
13         base = max(RBig(1) - delta - alpha, RBig(0))  
14         t2 = exp_neg_eps * base  
15         return max(max(t1, t2), RBig(0))  
16  
17     return tradeoff
```

Postcondition

Theorem 1.1. For all $\alpha \in [0, 1]$, the pseudocode returns $\tilde{f}(\alpha) \leq f_{\epsilon, \delta}(\alpha)$, where $f_{\epsilon, \delta}$ is the exact approximate-DP tradeoff curve.

We start with the following standard definition of approximate DP.

Definition 1.2. Let $\epsilon > 0$ and $\delta \geq 0$, and define

$$f_{\epsilon,\delta}(\alpha) = \max(0, 1 - \delta - \exp(\epsilon)\alpha, \exp(-\epsilon)(1 - \delta - \alpha)). \quad (1)$$

Then we say that a mechanism M satisfies (ϵ, δ) -DP if it satisfies $f_{\epsilon,\delta}$ -DP.

Proof. The definition assumes $\alpha \in [0, 1]$, which is the domain of the returned function. The code clamps the base of the second affine branch at zero, as in Definition 1.2.

The interval upper endpoint on line 8 over-estimates $\exp(\epsilon)$, and the interval lower endpoint on line 9 under-estimates $\exp(-\epsilon)$. The results of these constants are converted exactly into rationals to be used in the tradeoff function.

The function defined on line 11 implements the formula in Definition 1.2 with exact fractional arithmetic, except that it substitutes these conservative rounded constants for $\exp(\epsilon)$ and $\exp(-\epsilon)$.

For $\alpha \in [0, 1]$, the first branch

$$1 - \delta - \mathbf{exp_eps} \cdot \alpha$$

is no greater than

$$1 - \delta - \exp(\epsilon)\alpha,$$

because $\mathbf{exp_eps} \geq \exp(\epsilon)$ and $\alpha \geq 0$.

For the second branch, if $1 - \delta - \alpha \geq 0$, then

$$\mathbf{exp_neg_eps} \cdot (1 - \delta - \alpha) \leq \exp(-\epsilon)(1 - \delta - \alpha),$$

because $\mathbf{exp_neg_eps} \leq \exp(-\epsilon)$. If instead $1 - \delta - \alpha < 0$, then both expressions are negative, so they do not affect the outer maximum with 0.

Therefore, for all $\alpha \in [0, 1]$, the function returned by the pseudocode is pointwise no greater than $f_{\epsilon,\delta}(\alpha)$. This is the required reporting behavior because a smaller β is a weaker tradeoff guarantee. $\square$